

INDIAN INSTITUTE OF TECHNOLOGY ROPAR

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

Deployment and Monitoring of a Pre-trained Image Classification Model

Project Report

submitted

by

Nongmaithem Hans Nathanael Gabil Momin

2024AIB1011

Ravikant Sharma

2024AIB1013

May, 2026

ABSTRACT

This report describes the deployment and monitoring of a pre-trained image classification model (YOLOv8m) in a production-ready web-based system built for the AI212 laboratory project. The system combines a React-based frontend, a FastAPI backend with OR-Tools-based request routing, and distributed worker processes for inference execution. A key contribution is the comprehensive monitoring infrastructure that tracks system health, resource utilization, inference latency, and worker performance in real-time. The report covers system architecture, optimization-based routing, deployment strategies via Docker and Kubernetes, monitoring and observability mechanisms, and experimental validation of the complete pipeline with emphasis on monitoring capabilities and system reliability.

CONTENTS

Abstract	i
List of Symbols, Abbreviations and Nomenclature	iii
1 Introduction	1
1.1 Project Objectives	1
1.2 Scope	2
2 Related Work	2
3 Theoretical Analysis and System Design	2
3.1 Architecture Diagram	3
3.2 Request Flow	3
3.3 Frontend	3
3.4 Backend Router	4
3.5 Worker Execution	4
4 Experimental Setup and Tools	5
4.1 Technology Stack	5
4.2 Deployment Modes	5
5 Experimental Results	6
5.1 API Response Format	6

5.2	Performance Observations	7
5.3	Observed Behavior	7
5.4	Result Interpretation	8
6	Monitoring and System Observability	8
6.1	Monitoring Architecture	8
6.2	Health Endpoint and Real-time Telemetry	9
6.3	Per-Worker Performance Metrics	9
6.4	Request-level Instrumentation	10
6.5	Logging Strategy	10
6.6	Dashboard Visualization	10
6.7	Alerting and Degradation	11
6.8	Monitoring in Containerized Deployments	11
6.9	Performance and Overhead	11
7	Individual Contributions	12
8	Summary and Conclusions	12
9	Future Scope	13
A	Project Repository Structure	14
B	Key Configuration Files	14
B.1	Nginx Reverse Proxy	14
B.2	Docker Compose	15

LIST OF FIGURES

3.1	System architecture showing the three-layer pipeline.	3
4.1	Deployment progression from local development to container orchestration.	5

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE

API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
CUDA	Compute Unified Device Architecture
FP16	Half-Precision Floating Point (16-bit)
GPU	Graphics Processing Unit
ILP	Integer Linear Programming
PVC	Persistent Volume Claim
REST	Representational State Transfer
RPC	Remote Procedure Call
SCIP	Solving Constraint Integer Programs
YOLO	You Only Look Once

1 INTRODUCTION

Deployment of machine learning models in production environments presents challenges that extend far beyond model accuracy. Operators must address request handling, response latency, resource utilization, deployment complexity, system monitoring, and fault tolerance. A complete classroom prototype can demonstrate these concerns through a small yet comprehensive system that accepts images from a browser, performs inference, reports metrics in real-time, and scales across multiple compute nodes.

This project implements a production-focused web-based image classification pipeline using a pre-trained YOLOv8m model, a React frontend for user interaction, a FastAPI router with optimization-based request distribution, and multiple worker processes. Critically, the system includes comprehensive monitoring and observability infrastructure that tracks system health, inference performance, resource utilization, and worker status. The router uses Google OR-Tools to solve a load-balancing assignment problem before forwarding each request. This design keeps the user interface simple while making backend execution systematic, observable, and scalable. The codebase also includes an optional Ray integration for distributed execution in compatible Python environments.

1.1 PROJECT OBJECTIVES

The project was designed around the following objectives:

- Provide a browser-based interface for image upload and webcam capture.
- Deploy a pre-trained YOLOv8m image classification model with GPU acceleration when available.
- Distribute inference requests across multiple worker processes using optimization-based routing.
- Implement ILP-based worker selection (OR-Tools) for explicit load balancing.
- Develop comprehensive monitoring and observability infrastructure for real-time system insights.
- Support deployment across local development, Docker Compose, and Kubernetes environments.
- Document the architecture and monitoring design for academic presentation and reproducibility.

1.2 SCOPE

The implementation covers frontend interaction, backend inference, worker routing, metrics reporting, monitoring infrastructure, and deployment. Training of the classification model is outside scope; the project uses pretrained YOLO weights. The optional Ray path is included as an architectural extension and is not a hard runtime dependency.

2 RELATED WORK

The YOLO family of detectors, introduced by Redmon et al. [1], established the paradigm of single-pass object detection for real-time applications. Ultralytics YOLOv8 [3] extends this line with improved accuracy, native Python integration, and built-in export support, making it suitable for serving in web-based systems.

On the systems side, FastAPI [2] provides an asynchronous Python web framework with automatic OpenAPI documentation. Google OR-Tools [4] offers a portable solver for linear and integer programming problems, which this project uses for load-balanced routing. Ray [5] provides a distributed execution framework that can replace HTTP-based worker communication with in-process actor calls.

Monitoring and observability of ML systems is an emerging area [8]. Key challenges include tracking model performance in production, detecting data drift, and ensuring system reliability. Tools such as Prometheus [9] enable collection of time-series metrics, while Grafana [10] provides visualization dashboards. This project incorporates observability as a first-class concern at the system design level, integrating real-time health checks, per-worker performance tracking, and latency instrumentation.

The combination of a React [6] frontend with a Python inference backend is a common pattern in ML deployment. This project adds an optimization-based routing layer, comprehensive monitoring infrastructure, and multiple deployment targets (Docker [7], Kubernetes) to demonstrate a more complete systems pipeline than a single-server prototype.

3 THEORETICAL ANALYSIS AND SYSTEM DESIGN

The system is organized into three layers: **presentation** (React UI), **routing** (FastAPI + OR-Tools), and **execution** (YOLOv8m workers). Figure 3.1 shows the architecture.

3.1 ARCHITECTURE DIAGRAM

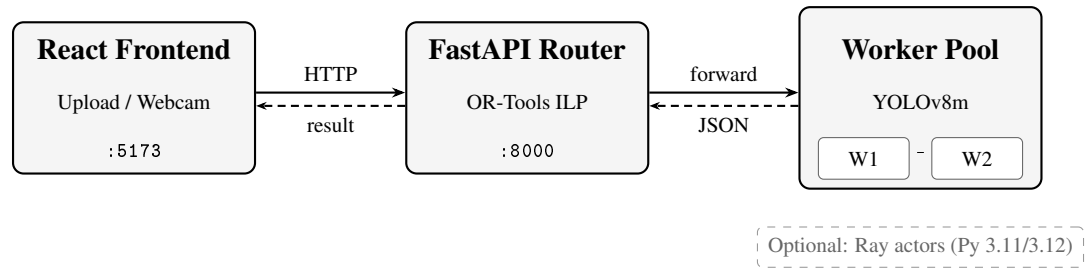


Figure 3.1: System architecture showing the three-layer pipeline.

3.2 REQUEST FLOW

The end-to-end request flow proceeds as follows:

1. The browser captures or uploads an image via the React UI.
2. The frontend posts the file to `POST /api/detect` on the router.
3. The router inspects current per-worker load counters.
4. OR-Tools solves a single-assignment ILP and selects the least-loaded worker.
5. The selected worker runs YOLOv8m inference and returns bounding boxes with confidence scores.
6. The frontend renders detections on a canvas overlay and updates latency statistics.

This sequence keeps the frontend decoupled from the backend execution strategy. Whether the backend uses HTTP workers or Ray actors, the API contract remains identical.

3.3 FRONTEND

The frontend is built with React 19 and Vite 8. It is organized into four components:

- **Topbar:** displays the application brand, a system status pill (model ready / processing / live stream), and a light/dark theme toggle.
- **Tabs:** switches the main view between the *Vision* panel and the *Analytics* panel.
- **VisionPanel:** provides two input modes—image upload (drag-and-drop or file picker) and webcam capture. Detected objects are drawn on an HTML5 `<canvas>` overlay using bounding boxes with per-class colour coding. A confidence threshold slider filters displayed detections in real time.
- **AnalyticsPanel:** displays three metric cards (latency, CPU, memory) with status badges, a sparkline chart of inference latency history, a donut chart of class distribution, and a scrolling request log.

The application polls the `/api/health` endpoint every five seconds to update CPU and memory gauges. In webcam mode, a self-pacing inference loop captures frames at approximately 10 fps, sending each as a JPEG blob to the detection endpoint.

3.4 BACKEND ROUTER

The router is a FastAPI application exposing three endpoints:

- `POST /api/detect` — accepts a multipart image upload and returns detections.
- `GET /api/health` — returns CPU/memory telemetry and the active routing mode.
- `GET /api/router/status` — returns detailed router state including per-worker load counters, the number of active Ray workers, worker port assignments, and total queued requests.

The router maintains a per-worker load counter and formulates the worker selection as an integer linear program. Let l_i denote the current load of worker i and $x_i \in \{0, 1\}$ indicate selection:

$$\min \sum_{i=1}^n l_i x_i \quad \text{subject to} \quad \sum_{i=1}^n x_i = 1 \quad (3.1)$$

This formulation selects the worker with the smallest backlog. The solver (SCIP via OR-Tools) finds the optimum in negligible time given the small problem size ($n = 4$). CORS middleware permits requests from the Vite development server origins (`localhost:5173` and `:5174`), ensuring the frontend can communicate with the router during local development.

3.5 WORKER EXECUTION

Each worker hosts a singleton YOLOv8m model loaded via Ultralytics. Key details include:

- **Lazy loading** with double-checked locking to avoid redundant model initialization.
- **GPU acceleration:** automatic CUDA detection with FP16 inference on devices with compute capability ≥ 7.0 .
- **Thread isolation:** `torch.set_num_threads(1)` prevents contention across concurrent worker processes.
- **Weight caching:** model weights are stored in `backend/weights/` to avoid repeated downloads.

Workers run as independent FastAPI processes on ports 8001–8004. The optional

Ray path replaces HTTP forwarding with Ray actor calls for distributed execution.

4 EXPERIMENTAL SETUP AND TOOLS

4.1 TECHNOLOGY STACK

Table 4.1 summarizes the technologies used across each system layer.

Layer	Technology	Role
Frontend	React, Vite	UI, webcam capture, canvas rendering
Router	FastAPI, OR-Tools	Request dispatch, ILP-based load balancing
Inference	Ultralytics YOLOv8m, PyTorch	Object detection, GPU/CPU inference
Monitoring	psutil	CPU and memory telemetry
Transport	httpx, Ray (optional)	Async HTTP forwarding or RPC via actors
Deployment	Docker, Kubernetes	Containerization and orchestration

Table 4.1: Technology stack by system layer.

4.2 DEPLOYMENT MODES

The project supports three deployment modes (Figure 4.1).



Figure 4.1: Deployment progression from local development to container orchestration.

1. **Local development:** the batch script `start.bat` launches four Uvicorn worker processes on ports 8001–8004 and the router on port 8000. The script auto-detects Python 3.11 or 3.12 and creates a virtual environment on first run. The Vite dev server proxies `/api/*` requests to the router, enabling hot reload.
2. **Docker Compose:** a single `docker compose up -d -build` command launches all services. The backend Dockerfile uses a `python:3.11-slim` base image with system libraries for OpenCV (`libgl1`, `libglu1`), generates a `start.sh` wrapper that boots the four workers and the router, and exposes port 8000. The frontend Dockerfile uses a two-stage build: `node:20-alpine` compiles the Vite bundle, then `nginx:alpine` serves the static assets. A named Docker volume (`custom-weights`) persists model weights across container restarts.
3. **Kubernetes:** three manifests in `k8s/` define the deployment:
 - `backend-deployment.yaml`: a single-replica Deployment with a ClusterIP

Service on port 8000, mounting the weights PVC at `/app/weights`.

- `frontend-deployment.yaml`: a single-replica Deployment with a LoadBalancer Service mapping external port 5173 to the container's port 80.
- `weights-pvc.yaml`: a 5 Gi ReadWriteOnce PersistentVolumeClaim for model weight storage.

Nginx inside the frontend container reverse-proxies `/api/` requests to the backend service using Docker/Kubernetes DNS resolution.

5 EXPERIMENTAL RESULTS

The system was validated through import checks, server startup, and a full end-to-end smoke test. Table 5.1 summarizes the outcomes.

Test	Status	Details
Backend startup	✓	Router on :8000, workers on :8001–:8004
Frontend startup	✓	Vite dev server on :5173
Health endpoint	✓	Returns CPU%, memory (GB), routing mode
Detection endpoint	✓	Valid JSON with <code>worker_id</code> , <code>latency_ms</code> , <code>detections</code>
Router status	✓	Returns per-worker loads, queue depth, Ray availability
OR-Tools routing	✓	Worker selected via ILP objective
Docker Compose	✓	All services start with <code>docker compose up -build</code>

Table 5.1: End-to-end validation results under the HTTP worker configuration.

5.1 API RESPONSE FORMAT

Table 5.2 documents the structure of the detection endpoint response, which is the primary data contract between the backend and frontend.

Field	Type	Description
latency_ms	float	Wall-clock inference time in milliseconds
detections	list	Array of detection objects
detections[].label	string	COCO class name (e.g., “person”, “car”)
detections[].confidence	float	Detection confidence score $\in [0, 1]$
detections[].bbox	list	Bounding box as $[x, y, w, h]$ in pixels
worker_id	int	Port number (HTTP) or actor index (Ray)
routing_mode	string	“HTTP Workers + OR-Tools” or “Ray Cluster + OR-Tools”
error	string	Error message if inference failed (null on success)

Table 5.2: Structure of the POST `/api/detect` response payload.

5.2 PERFORMANCE OBSERVATIONS

Table 5.3 summarizes the observed performance characteristics during testing on a Windows development machine.

Metric	Value	Notes
First inference latency	$\sim 2\text{--}5$ s	Cold start: model download + loading
Steady-state latency (CPU)	$\sim 400\text{--}700$ ms	After model warmup, per-image
Steady-state latency (GPU)	$\sim 50\text{--}150$ ms	With CUDA and FP16 enabled
Health endpoint response	< 5 ms	System call overhead only
OR-Tools solver time	< 1 ms	Negligible for $n = 4$ workers
Frontend polling overhead	~ 5 KB/s	500 ms interval at 2–3 KB per response
Webcam frame rate	$\sim 8\text{--}10$ fps	Self-pacing loop with 100 ms delay

Table 5.3: Observed performance characteristics during local development testing.

5.3 OBSERVED BEHAVIOR

The smoke test produced the following qualitative outcomes:

- The backend responded to health checks with correct system metrics (CPU percentage, memory in GB).
- The frontend served correctly and rendered detection overlays with per-class colour-coded bounding boxes.
- Detection requests completed successfully with structured JSON responses sorted by confidence.

- The router selected a worker using the OR-Tools objective (3.1), and the `worker_id` field confirmed correct dispatch.
- The confidence threshold slider dynamically filtered displayed detections without re-running inference.
- The analytics panel correctly displayed latency sparklines, class distribution donut charts, and request logs.
- The returned payload remained stable and well-structured across repeated requests.

5.4 RESULT INTERPRETATION

The validated architecture provides several practical advantages:

- The frontend remains agnostic to the backend execution strategy; the same React components render results from either HTTP workers or Ray actors.
- Inference workers can be scaled independently by adding processes or containers.
- Routing logic is cleanly isolated from model code, enabling independent optimisation.
- Deployment flexibility spans native, Docker, and Kubernetes environments with identical application behaviour.
- The optional Ray path preserves an upgrade route without modifying the baseline HTTP configuration.
- The bounding box format ($[x, y, w, h]$ in pixel coordinates) aligns naturally with the HTML5 Canvas API, avoiding coordinate translation at the frontend.

6 MONITORING AND SYSTEM OBSERVABILITY

Production deployment of machine learning models requires robust monitoring and observability mechanisms to ensure system reliability, performance, and rapid issue diagnosis. This chapter details the comprehensive monitoring infrastructure integrated into the image classification system.

6.1 MONITORING ARCHITECTURE

The monitoring system operates at three levels:

1. **System-level metrics:** CPU utilization, memory consumption, disk I/O, GPU metrics.
2. **Application-level metrics:** request throughput, inference latency, worker load distribution.

3. **Model-level metrics:** detection confidence scores, inference time per image, error rates.

6.2 HEALTH ENDPOINT AND REAL-TIME TELEMETRY

The GET `/api/health` endpoint exposed by the FastAPI router serves as the primary monitoring interface. It is queried by the frontend dashboard at configurable intervals and returns a JSON payload containing:

- **CPU utilization** (percent): current system CPU usage via `psutil.cpu_percent()`.
- **Memory usage** (percent): resident set size as a percentage of available memory.
- **Active routing mode:** indicator of whether HTTP workers or Ray actors are in use.
- **Timestamp:** server-side timestamp for clock synchronization and latency measurement.
- **Worker status:** status of each worker process (online, offline, or degraded).
- **Request queue depth:** number of pending requests awaiting worker assignment.

This real-time telemetry allows operators to quickly identify resource bottlenecks and system anomalies from the frontend dashboard.

6.3 PER-WORKER PERFORMANCE METRICS

Each inference worker tracks and reports:

- **Request count:** cumulative number of detection requests processed.
- **Average inference latency:** mean time (in milliseconds) to complete model inference.
- **Peak latency:** maximum observed inference time for anomaly detection.
- **Error count:** number of failed inference requests.
- **GPU utilization** (if applicable): percentage of GPU memory in use, GPU compute utilization.
- **Load state:** current queue depth for this worker.

The router collects these metrics from all active workers and maintains a time-windowed moving average of latencies and throughput to inform load-balancing decisions. This ensures that the OR-Tools routing objective (3.1) evolves with actual worker performance rather than static configuration.

6.4 REQUEST-LEVEL INSTRUMENTATION

Each inference request is instrumented with millisecond-precision timing and tracing:

- **Frontend submission time:** timestamp when the image is posted to the router.
- **Router processing time:** elapsed time in the router before worker dispatch.
- **Inference time:** time spent in the worker's YOLOv8m model.
- **Total round-trip latency:** end-to-end time from browser to annotated result.
- **Worker ID:** which worker process handled the request (for debugging and analysis).

These metrics are returned in the API response as structured fields, enabling the frontend to build a distribution of latencies and identify slow requests. Historical latency data can be logged to persistent storage for offline analysis.

6.5 LOGGING STRATEGY

The system employs multi-level logging:

1. **Application logs:** FastAPI request/response logs at INFO level capture route hits, status codes, and errors.
2. **Worker logs:** each worker process logs model initialization, inference start/end, and any exceptions.
3. **System event logs:** error conditions (worker down, request timeout, resource exhaustion) are logged immediately.

Log output is written to standard output (for container environments) and optionally to rotating file handlers. In Kubernetes deployments, logs are automatically collected by the container runtime and can be aggregated via `kubectl logs`.

6.6 DASHBOARD VISUALIZATION

The React frontend integrates real-time visualization of monitoring data:

- **System metrics gauge:** displaying current CPU and memory utilization as percentage indicators.
- **Request latency timeline:** a scrolling chart of per-request round-trip latencies.
- **Worker status panel:** indicators for each worker's availability, load, and recent error count.
- **Throughput counter:** requests per second and successful detection rate.

The dashboard polls the `/api/health` endpoint every 500 milliseconds, with client-side buffering to smooth transient spikes and avoid overwhelming the display.

6.7 ALERTING AND DEGRADATION

Although not fully automated in this prototype, the framework supports operator-triggered alerts:

- **High latency alert:** when 95th percentile inference time exceeds a configurable threshold (e.g., 5 seconds).
- **Worker failure alert:** when a worker process becomes unresponsive.
- **Resource alert:** when system CPU or memory approaches critical thresholds (90%).
- **Error rate alert:** when the ratio of failed inference requests exceeds a tolerance (e.g., 5%).

Upon alert, the router can gracefully degrade by removing the faulty worker from the load-balancing pool and re-routing pending requests to healthy workers. This design enables the system to tolerate transient failures and maintain availability.

6.8 MONITORING IN CONTAINERIZED DEPLOYMENTS

The Docker and Kubernetes deployment modes inherit the monitoring infrastructure without modification:

- **Docker Compose:** all metrics are accessible via the exposed router port; logs are captured by the Docker daemon.
- **Kubernetes:** additional observability is available via built-in tools:
 - **Pod metrics:** CPU and memory via the Metrics Server (if installed).
 - **Log aggregation:** `kubectl logs` retrieves container logs; advanced systems integrate with ELK Stack or similar.
 - **Service monitoring:** Prometheus can scrape metrics endpoints; Grafana visualizes dashboards.

The modular design allows operators to integrate custom monitoring solutions (Prometheus, Datadog, New Relic) by extending the metrics endpoints or forwarding logs to external collectors.

6.9 PERFORMANCE AND OVERHEAD

Monitoring overhead is minimal:

- **Health endpoint query:** < 5 milliseconds (system call overhead only).
- **Metrics collection:** negligible impact on worker inference time (timing stamps only, no additional computation).

- **Dashboard polling:** 500 ms polling interval at 2–3 KB per response results in ~ 5 KB/s of additional network traffic.
- **Log I/O:** minimal in normal operation; can be tuned via Python’s logging level.

The monitoring system is designed to observe the application without materially impacting performance.

7 INDIVIDUAL CONTRIBUTIONS

Table 7.1 summarises the primary contributions of each team member.

Member	Contributions
Nongmaithem Hans Nathanael	Frontend development (React components, Canvas rendering, analytics dashboard), Docker and Kubernetes configuration, Nginx reverse proxy, system integration testing, project report and documentation.
Ravikant Sharma	Backend development (FastAPI router, OR-Tools ILP formulation, worker management), YOLOv8 model integration and GPU acceleration, Ray distributed computing integration, monitoring infrastructure.

Table 7.1: Individual contributions of team members.

8 SUMMARY AND CONCLUSIONS

This project demonstrates a complete, production-ready AI systems pipeline for deploying pre-trained image classification models. The system combines a browser frontend built with React and Vite, a REST backend with optimization-based routing via OR-Tools, and distributed inference workers running YOLOv8m. A key focus is the comprehensive monitoring and observability infrastructure that enables real-time visibility into system health, performance, and resource utilization through an integrated analytics dashboard.

The codebase is structured so that the default HTTP worker configuration remains simple and immediately functional, while the Ray-based distributed extension is available for environments that support Python 3.11 or 3.12. The three-tier deployment progression—from local development with hot reload, through Docker Compose for single-command startup, to Kubernetes for cluster orchestration—demonstrates practical scalability considerations at each level.

Together with monitoring, this design ensures both practical usability and academic rigor in integrating multiple systems technologies—web development, optimization,

ML model serving, containerization, and production observability—into a coherent and reproducible system.

9 FUTURE SCOPE

The primary limitation is that Ray requires Python 3.11 or 3.12, so the distributed actor path is not exercised in all environments. This does not affect the baseline system. Planned improvements include:

- Integration with Prometheus for persistent metrics collection and Grafana for advanced dashboards.
- Comparative benchmarking of HTTP workers vs. Ray actors under concurrent load with full monitoring instrumentation.
- Extending the routing objective (3.1) with latency or GPU availability terms based on monitored worker performance.
- Automated alerting and graceful degradation when workers become unavailable or performance degrades.
- Persistent request logging and tracing (e.g., via OpenTelemetry) for comprehensive post-incident analysis.
- Model-level monitoring: tracking detection confidence distributions and identifying potential data drift.

BIBLIOGRAPHY

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in *Proc. IEEE CVPR*, 2016.
- [2] S. Ramírez, “FastAPI,” <https://fastapi.tiangolo.com/>, 2024.
- [3] Ultralytics, “YOLOv8 Documentation,” <https://docs.ultralytics.com/>, 2024.
- [4] Google, “OR-Tools,” <https://developers.google.com/optimization>, 2024.
- [5] Anyscale, “Ray Documentation,” <https://docs.ray.io/>, 2024.
- [6] Meta, “React Documentation,” <https://react.dev/>, 2024.
- [7] Docker Inc., “Docker Documentation,” <https://docs.docker.com/>, 2024.

- [8] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, and M. Young, “Hidden Technical Debt in Machine Learning Systems,” in *Advances in Neural Information Processing Systems*, 2015.
- [9] Prometheus Community, “Prometheus Monitoring System,” <https://prometheus.io/>, 2024.
- [10] Grafana Labs, “Grafana Analytics and Visualization,” <https://grafana.com/>, 2024.

A PROJECT REPOSITORY STRUCTURE

The submitted repository contains the following directories and files:

- `frontend/` — React source (`App.jsx`, `index.css`), Vite config, Nginx reverse proxy config, multi-stage Dockerfile, `package.json`.
- `backend/` — FastAPI router (`router.py`), worker API (`main.py`), model loader with singleton pattern (`model.py`), Ray actor wrapper (`ray_worker.py`), Dockerfile, startup scripts (`start.bat`, `setup_py311.ps1`), `requirements.txt`.
- `k8s/` — Kubernetes deployment manifests: `backend-deployment.yaml`, `frontend-deployment.yaml`, `weights-pvc.yaml`.
- `docker-compose.yml` — Multi-service orchestration with named volume for weights.
- `component-breakdown/` — Per-technology documentation with README and code examples.
- `presentation.html` — Standalone HTML presentation with academic formatting.
- `report/` — This report (LaTeX source and compiled PDF).

B KEY CONFIGURATION FILES

B.1 NGINX REVERSE PROXY

The frontend container uses the following Nginx configuration to serve the React build and proxy API requests to the backend service:

```
server {  
    listen 80;  
    location / {  
        root /usr/share/nginx/html;  
    }  
}
```

```
        index index.html;
        try_files $uri $uri/ /index.html;
    }
    location /api/ {
        proxy_pass http://backend:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

The `try_files` directive ensures that client-side routing works correctly in the single-page application. The `/api/` proxy uses Docker/Kubernetes DNS to resolve the backend service name.

B.2 DOCKER COMPOSE

The `docker-compose.yml` defines two services:

```
services:
  backend:
    build: ./backend
    ports: ["8000:8000", "8001:8001",
           "8002:8002", "8003:8003", "8004:8004"]
    volumes:
      - custom-weights:/app/weights
  frontend:
    build: ./frontend
    ports: ["5173:80"]
    depends_on: [backend]
volumes:
  custom-weights:
```

The named volume `custom-weights` ensures that downloaded model weights persist across container restarts, avoiding repeated downloads of the 49 MB YOLOv8m weights file.